

Odabrana poglavlja iz operativnih sistema

Projektni zadatak

Pridržavati se principa objektno-orijentisanog programiranja, SOLID principa, principa pisanja čistog koda i konvencija korištenih programskih jezika. Projektni zadatak se sastoji iz dvije stavke. Prilikom predaje potrebno je predati sve *.py* fajlove, *requirements.txt* fajl u kom su navedeni nazivi i verzije biblioteka, kao i dodatne fajlove ukoliko su rađene dodatne stavke koje to zahtijevaju.

(23) PageRank algoritam

[PageRank](#) je algoritam za rangiranje čvorova u usmjerenom grafu (npr. web stranice sa linkovima, mreža citata između radova, mreža praćenja između korisnika). Razvijen je od strane Googla za potrebe Google pretrage i danas predstavlja jedan od osnovnih algoritama koji je pokreće. Osnova algoritma se zasniva na ideji da je neki čvor “važniji” ako na njega pokazuje mnogo drugih čvorova, posebno ako ti čvorovi već imaju visoku važnost.

U standardnom modelu, zamišlja se “nasumični surfer” koji:

- sa vjerovatnoćom d (damping faktor, tipično 0.85) prati jednu od izlaznih veza trenutnog čvora,
- sa vjerovatnoćom $(1 - d)$ “teleportuje” se na slučajan čvor (ili na čvor iz definisanog skupa, u personalizovanom slučaju).

Osnovne komponente algoritma u distribuiranom kontekstu (RDD):

1. Inicijalizacija: svim čvorovima dodijeliti početni rang (najčešće $1/N$, gdje je N broj čvorova).
2. Iteracije (superstepovi): u svakoj iteraciji svaki čvor “raspodijeli” svoj trenutni rang na svoje izlazne susjede:
 - ako čvor ima k izlaznih veza, svakom susjedu šalje doprinos $\text{rank}(u)/k$
 - svi doprinosi koji pristižu u isti čvor se saberu (reduce)
3. Ažuriranje - novi rang čvora v izračunati kao:
$$\text{newRank}(v) = (1 - d) / N + d * (\text{sumIncoming}(v) + \text{danglingMass} / N),$$

gdje je danglingMass suma rangova čvorova koji nemaju nijednu izlaznu vezu ($\text{outDegree} = 0$).
Ova masa se ravnomjerno raspodjeljuje na sve čvorove.
4. Kriterijum zaustavljanja - izvršavati iteracije dok:
 - se rangovi ne stabilizuju (npr. ukupna promjena $|\text{newRank} - \text{oldRank}|$ ispod epsilon), ili
 - se ne dostigne maksimalan broj iteracija (maxIter).
5. Rezultat: rangovi svih čvorova (i top-K najvažnijih čvorova).

Napomena: Za sve operacije nad grafom potrebno je koristiti isključivo Spark RDD API! Nije dozvoljena obrada nad podacima nakon poziva *.collect()* funkcije (osim eventualno za preuzimanje top-K rezultata). Sve Spark lambda funkcije treba da rade maksimalno nad jednim čvorom i njegovim lokalnim podacima (npr. njegova lista izlaznih susjeda i/ili njegov rang), bez učitavanja cijelog grafa u memoriju drajvera.

(3) Implementirati klasu koja reprezentuje čvor usmjerenog grafa. Svaki čvor mora imati:

- ID (npr. int),
 - listu izlaznih susjeda (outNeighbors: list[int]),
 - izlazni stepen (outDegree), koji se može računati iz liste ili čuvati kao atribut.
- Po potrebi dozvoljeno je uključiti i dodatne attribute (npr. težine veza, meta-podatke za testiranje).

Implementirati funkciju/skriptu za kreiranje skupa podataka (sintetički graf) koja prima:

- broj čvorova N,
- broj veza E (ili prosječan izlazni stepen),
- seed za determinističko generisanje.

Generisani graf mora biti: usmjeren, bez petlji (nema (u,u)), i bez duplikata veza (u,v) koliko je moguće (dozvoljeno je očistiti duplikate RDD operacijama).

Pored generisanja, omogućiti i učitavanje grafa iz edge-list fajla (npr. CSV/TSV format: "srcId dstId"). Omogućiti serijalizaciju i deserijalizaciju grafa upotrebom JSON formata. Prilikom serijalizacije i deserijalizacije koristiti samo osnovne attribute (ID i lista izlaznih susjeda).

(6) Implementirati PageRank algoritam upotrebom Spark RDD API-ja, u skladu sa opisanim koracima. Minimalni zahtjevi:

- d damping faktor kao parametar (podrazumijevano 0.85),
- maxIter kao parametar,
- epsilon kao parametar (kriterijum stabilizacije),
- ispravno rukovanje "dangling" čvorovima (outDegree = 0) preko danglingMass/N,
- računanje i čuvanje finalnih rangova (nodeId, rank).

Prilikom izvršavanja algoritma:

- rangovi se moraju ažurirati iterativno,
- u svakoj iteraciji potrebno je izračunati sumIncoming doprinose i danglingMass,
- nakon završetka ispisati i/ili sačuvati top-K čvorova po rangui (K kao parametar).

(3) Implementirati funkciju za validaciju rezultata koja se oslanja isključivo na Spark RDD API. Validacija treba da provjeri bar sljedeće:

- svi rangovi su nenegativni,
- suma rangova je približno 1 (tolerancija npr. $1e-3$ ili $1e-4$),
- broj rangova odgovara broju čvorova,
- dangling čvorovi su ispravno tretirani (npr. testirati na malom grafu sa poznatim slučajem).

Napomena: za validaciju nije dozvoljeno učitavanje cijelog rezultata u drajver (osim eventualno nekoliko agregata i top-K).

(2) Implementirati funkciju koja omogućava praćenje vremena izvršavanja algoritma (ukupno vrijeme i po iteraciji). Izložiti CLI interfejs za pokretanje programa. Dozvoljeno je koristiti argparse ili eksterne biblioteke ([click](#), [fire](#)). Interfejs treba da omogući sljedeće funkcionalnosti:

- Specifikovanje ulazne datoteke edge-list grafa ili definisanje parametara generisanja (N, E, seed)
- Definisanje izlaznog fajla za serijalizaciju grafa (opciono)
- Definisanje izlaznog fajla za finalne rangove (nodeId, rank)
- Podešavanje d, maxIter, epsilon i K (top-K)

(5) Optimizovati brzinu izvršavanja algoritma. Optimizacija mora da ostvari kraće vrijeme izvršavanja u odnosu na baznu implementaciju. Prilikom optimizacije koristiti neke od sljedećih optimizacija:

- Optimizacija inicijalnog particionisanja
- Upotreba boljeg keširanja međurezultata
- Optimizacija i redukcija shuffle-inga

Za datu modifikaciju potrebno je priložiti .pdf fajl u kom su opisane modifikacije, te priloženo teoretsko obrazloženje njihovog pozitivnog uticaja na rad algoritma. Takođe, priložiti tabelu u kojoj je upoređena brzina između 5 pokretanja algoritma na grafovima različite veličine i broja susjeda (uporediti minimalno na 2 grafa, manji i veći broj čvorova).

(4) Implementirati personalizovani PageRank (Personalized PageRank). Umjesto uniformnog teleportovanja na bilo koji čvor, teleportovanje je usmjereno na zadati skup “seed” čvorova S (npr. korisnikova interesovanja, početni izvori u analizi, preferirane stranice).

Za personalizovani PageRank:

- teleport komponenta raspodjeljuje $(1 - d)$ vjerovatnoću samo na čvorove iz S (npr. ravnomjerno unutar S, a 0 van S),
- ostali dio ($d * \text{link-following}$) ostaje isti kao u standardnom PageRank-u,
- S treba da se zada kroz CLI (lista ID-eva ili putanja do fajla sa ID-evima).

Potrebno je izračunati i uporediti:

- top-K čvorova za standardni PageRank,
- top-K čvorova za personalizovani PageRank,
- mjeru sličnosti rezultata (npr. preklapanje top-K).

Pokrenuti izvršavanje za 10 različitih nasumično generisanih grafova i izračunati procenat preklapanja rezultata između dva algoritma (rezultate sačuvati u fajl u JSON formatu).

(27) Analiza Citi Bike skupa podataka

(4) Data stavka projektnog zadatka će obuhvatiti analizu Citi Bike skupa podataka koristeći Spark DataFrame API. Za realizaciju datog zadatka potrebno je prvo definisati tačnu šemu podataka koja koristi adekvatne tipove na osnovu opisa podataka. Validirati da se podaci adekvatno učitavaju. Izvršiti osnovno čišćenje (npr. nekorektni vremenski intervali, nedostajuće ključne vrijednosti, ekstremne vrijednosti trajanja). Informacije o podacima se nalaze na sljedećim linkovima:

- [šema podataka](#)
- [podaci za 10.2024 koji se koriste za analizu](#)

Napomena: Koristiti isključivo Spark DataFrame API za sve analize podataka! Nije dozvoljeno rješavanje zadataka tako što se značajan dio podataka preuzme na driver (collect) i obrađuje lokalno (osim za male rezultate poput top-10/top-20). Sve podatke nakon analize je potrebno sačuvati u JSON ili .csv formatu (u slučaju velike količine podataka sačuvati samo prvih 100 rezultata u fajl).

(3) Kreirati izvedene kolone koje će se koristiti u daljim analizama, minimalno:

- trajanje vožnje (u sekundama/minutama),
- vremenske karakteristike početka vožnje (sat u danu, dan u sedmici, indikator vikenda),
- indikator “round trip” (početna i krajnja stanica ista),
- jedna dodatna inženjerska karakteristika koja koristi postojeće attribute (npr. približna udaljenost na osnovu koordinata, ili izvedena klasifikacija tipa vožnje), uz kratko objašnjenje šta ta karakteristika predstavlja.

(2) Analizirati kako se broj vožnji i “tipično” trajanje mijenjaju po satu u danu, odvojeno za radne dane i vikend, i odvojeno po *member_casual*. Rezultat prikazati u tabeli sa jasno naznačenim grupisanjima i sortiranjima.

(2) Identifikovati najaktivnije start stanice (top-N). Za izabrane stanice izračunati dodatne metrike koje opisuju profil stanice (npr. udio članova, udio round-trip vožnji, tipično trajanje, raspodjela kroz doba dana). Prikazati rang listu po najmanje jednoj od izvedenih metrika, uz prag minimalnog broja vožnji radi stabilnosti rezultata.

(2) Analizirati najčešće relacije start->end (top-N) uz uslov minimalnog broja vožnji. Za te relacije uporediti tipično trajanje i varijabilnost trajanja (odabrati odgovarajuću mjeru). U rezultatima jasno navesti relaciju, broj vožnji i izabrane metrike.

(2) Analizirati round trip vožnje: udio u ukupnom broju vožnji, tipično trajanje i način na koji se round trip ponašanje razlikuje po *member_casual* i po *rideable_type*. Rezultat prikazati tako da se razlike mogu direktno uporediti.

(4) Učitati Meteostat hourly podatke za [stanicu KJRB0 \(2024\)](#) i pripremiti dataset za spajanje sa Citi Bike podacima. Informacije o formatu Meteostat skupa podataka se nalaze na sljedećem [linku](#). Potrebno je:

- izvesti vremenski “bucket” pogodan za join (npr. sat),

- ograničiti weather podatke na period koji pokriva Citi Bike dataset (oktobar 2024),
- izvesti bar dvije vremenske karakteristike koje ćete koristiti u kombinovanoj analizi (npr. indikator padavina, temperaturni bandovi, bandovi vjetra), uz kratko objašnjenje definicije (može u komentaru).

(4) Nakon spajanja Citi Bike i vremenskih podataka po vremenskom “bucket”-u, analizirati uticaj padavina na potražnju i ponašanje: uporediti broj vožnji i tipično trajanje u “rainy” i “non-rainy” uslovima uz bar jednu kontrolu koja izbjegava trivijalne zaključke (npr. poređenje unutar istog sata u danu, ili odvojeno po vikend/radni dan, ili odvojeno po *member_casual*). Prikazati rezultate u tabelarnom obliku sa jasno naznačenim grupisanjima.

(4) Nakon spajanja Citi Bike i vremenskih podataka po vremenskom “bucket”-u, analizirati kako se potražnja i ponašanje mijenjaju kroz različite kategorije temperature. Meteostat koristi metričke jedinice (temperatura u °C) i vrijeme u UTC, pa je potrebno obratiti pažnju na konzistentnost vremena prije spajanja.

Temperaturne kategorije definisati koristeći sljedeće intervale (granice su donje-uključive, gornje-isključive, osim zadnjeg intervala):

- B1: vrlo hladno - $\text{temp} < 5^{\circ}\text{C}$
- B2: hladno - $5^{\circ}\text{C} \leq \text{temp} < 10^{\circ}\text{C}$
- B3: umjereno - $10^{\circ}\text{C} \leq \text{temp} < 15^{\circ}\text{C}$
- B4: toplo - $15^{\circ}\text{C} \leq \text{temp} < 20^{\circ}\text{C}$
- B5: vrlo toplo - $\text{temp} \geq 20^{\circ}\text{C}$

Za svaku kategoriju prikazati (minimalno) broj vožnji i vrijeme trajanja vožnji uz obaveznu segmentaciju po *member_casual* ili po *rideable_type* (može i oboje, ali nije obavezno). Rezultat prikazati u tabelarnom obliku sa jasno naznačenim grupisanjima i sortiranjima.